Refinement Invalidation under Mutation

Deep Dive F for "Refinement and Inference Growth in SKJS"

Models revision `9108f339` of `main` (2026-07-02)

Abstract

The main soundness paper proves *one-step branch soundness*: a condition is evaluated against a fixed store, and the refined environment is guaranteed only at that instant. Its assumption ledger records **L-PURE** as VIOLATED (issues #597, #599) and a tracked instance of the **L-MUT** boundary (issue #598). This deep dive is part formalization, part design document. It extends the one-step store model with mutation events, classifies the tracked unsoundness shapes and pins where each breaks the main paper's branch-soundness argument, establishes by code inspection and by running the checker what the implementation does at every mutation site (including two findings beyond the tracked issues: assignments to captured locals inside closures are *entirely unchecked*, and closure bodies *read* enclosing narrowings that may be stale at invocation), analyzes four invalidation policies, and recommends one — capture-based invalidation for locals plus field re-widening at writes and call-boundary havoc for field narrowings — together with the preservation-style lemma the chosen policy validates.

Contents

1 Purpose and Ledger Rows Addressed **2**

2 Minimal Recap of the Main Paper **2**

3 A Store Model with Mutation Events **3**
3.1 The three tracked shapes 4

4 What the Implementation Does at Each Mutation Site **5**
4.1 (M-Asgn) direct assignment to a local: sound flow update 5
4.2 (M-Asgn) through a closure: unchecked and invisible 5
4.3 (M-Field) syntactic field write: checked, but no update 6
4.4 (M-Call): no invalidation whatsoever 6

5 Design Space **6**
5.1 Option (i): havoc-on-call 6
5.2 Option (ii): effect / purity tracking 7
5.3 Option (iii): capture-based invalidation 7
5.4 Option (iv): status quo (documented tsc-parity unsoundness) 8
5.5 Recommendation 8

6 The Re-Widening Rule for Field Writes (#598) **9**

7 Status and Registry Impact **10**

1 Purpose and Ledger Rows Addressed

This paper addresses the following rows of the main paper's Assumptions and Proof Obligations table (its §8):

- **L-PURE** (VIOLATED) — "evaluating a condition does not mutate the narrowed locals", and, by extension, the unstated companion assumption that the store does not change between the condition and the *uses* of the narrowed environment. Tracked as issues #597 and #599.
- **L-MUT** (ASSUMED, with a tracked instance) — field writes are governed by set-types; the tracked instance is issue #598: a field reassignment inside a refined branch does not re-widen the narrowed field.
- **L-LOCAL** (PROVED for the module-level half) — the captured-local half, which the main paper says "sharpens L-PURE", is made precise here (Findings 4 and 5).

The deliverable is a *specified* invalidation policy (Policy 6) with a soundness statement (Lemma 7), plus the exact ledger-row edits the main paper should adopt (Section 7). L-PURE remains VIOLATED until the policy lands; this paper turns it from an open design question into a fix with a stated lemma.

2 Minimal Recap of the Main Paper

We reuse the main paper's notation without redefining it: values Val with runtime tags, object values ObjVal with the snapshot interface ($\mathsf{props}, \mathsf{get}, \ldots$), type denotations $[\![\cdot]\!]_\rho$ indexed by the substitution state ρ, environments Γ mapping locals to `TType`, stores σ mapping locals to values, satisfaction $\sigma \models_\rho \Gamma$, the evaluation relations $\sigma \models_b e$, and branch soundness for rules $\Gamma \vdash e \overset{b}{\rightsquigarrow} \Gamma'$ (main paper, Definitions "Environments and Stores" through "Branch Soundness"). Three recap points are load-bearing here:

1. **One-step scoping.** The main paper's branch-soundness theorems say: if $\sigma \models_\rho \Gamma$ and $\sigma \models_b e$ then $\sigma \models_{\rho'} \Gamma'$ — for the *same* σ. No claim is made about later program points; assignment, loops, and joins are covered only by merge soundness (its Lemma "Merge Soundness"). The gap between "Γ' holds at branch entry" and "Γ' holds at each use inside the branch" is exactly the subject of this paper.
2. **Snapshot object semantics.** Membership $o \in [\![\mathsf{object}(B)]\!]$ constrains the properties of o "at the moment a condition is evaluated" (main paper, Definition "Object Interface"), via get-types only; set-types govern which writes the checker accepts (**L-MUT**) and do not participate in membership.
3. **The refinement primitive writes to the scope.** All narrowing lands in the flow-sensitive scope by `setLocalFilterMap`/`scope.set`; field narrowing (R-EqField, R-TruthyField) rewrites the field's get inside the local's object type via `narrowField` → `updateField` → `TField::withGet`, leaving set untouched.

impl: `TypeCheckRefine.sk / setLocalFilterMap, narrowField; TType.sk / TField::withGet`

The scope chain. Because the invalidation question is about *where* flow information lives, we fix the implementation's scope discipline once. A `Scope` is a mutable frame with an `up` pointer, a per-frame `isLoop` flag, declared names (`locals`), and flow entries (`refs`). Lookup (`maybeGetRef`) walks the `up` chain and records whether it crossed a loop frame. The central asymmetry: `Scope::set` *always writes into the current frame*, even when the binding was found higher up — flow updates shadow rather than mutate the defining frame — and if the lookup crossed a loop frame, the assigned type is first checked against the type the binding had at loop entry (the back-edge guard). Structured constructs reconcile shadow frames explicitly: `if/else` via `mergeScope` (pointwise union) or `popScopeReplace` when one side terminates; loop and `try` frames via `popScope` (union with the parent entry). Closure bodies are the exception and the subject of Section 4.

impl: `TypeCheckScope.sk / Scope::set, maybeGetRefHelper, merge, popMerge, popReplace; TypeCheckStmt.sk / If, While, For arms`

3 A Store Model with Mutation Events

The main paper's store σ maps locals to values and is fixed. To talk about invalidation we split it and let it step.

Definition 1 (Two-Level Store)**.** *A store is a pair $\sigma = (\sigma_L, \sigma_H)$ where $\sigma_L : \mathsf{Var} \rightharpoonup \mathsf{Val}$ maps locals to values and σ_H is a heap giving each object identity its current property map; the snapshot interface $\mathsf{get}(o, p)$ of the main paper's the Object Interface definition is read as $\mathsf{get}_{\sigma_H}(o, p)$. Satisfaction $\sigma \models_\rho \Gamma$ means $\sigma_L(x) \in [\![\Gamma(x)]\!]_\rho^{\sigma_H}$ for all $x \in \mathrm{dom}(\Gamma)$, where object-type membership consults σ_H. All results of the main paper hold verbatim for a fixed (σ_L, σ_H).*

Definition 2 (Mutation Events)**.** *Between two checker-visible program points the store may step $\sigma \xrightarrow{\mu} \sigma'$ by one of:*

- ***(M-Asgn)** direct assignment $x := v$: replaces $\sigma_L(x)$; σ_H unchanged.*
- ***(M-Field)** field write $o.f := v$: replaces one property in σ_H; σ_L unchanged. Reachable from the checked function as `x.f = e`, or from* any *called code holding an alias of o. If f is accessor-backed, the event is instead "run the setter", which may perform arbitrary further events.*
- ***(M-Call)** call/new/await boundary: runs foreign code, which may perform any sequence of (M-Field) events on heap objects reachable from its arguments, from globals, or from closures it can reach, and any (M-Asgn) events on locals* captured by such closures*. Locals not captured by any closure are unreachable: (M-Call) cannot write them.*

JS is single-threaded per agent, so between two adjacent checker-visible points with no intervening call and no intervening syntactic write, the store is genuinely fixed; this is what makes the main paper's one-step theorems non-vacuous.[1]

Definition 3 (Preservation Discipline)**.** *The checker tracks a current environment Γ_t at each program point t. Call an environment-update function U_μ (what the checker does to Γ when it sees, or must conservatively assume, event μ)* adequate *for μ if $\sigma \models_\rho \Gamma$ and $\sigma \xrightarrow{\mu} \sigma'$ imply $\sigma' \models_\rho U_\mu(\Gamma)$. Flow-sensitive checking is sound iff every event that can occur between two points is matched by an*

[1] In a multi-agent (shared-memory) setting even that fails for `SharedArrayBuffer`-backed state; SKJS, like tsc, ignores this, and so do we.

adequate update on that path. The main paper's branch-soundness results supply the U for condition evaluation (no event, $\sigma' = \sigma$); this paper supplies — or exhibits the absence of — the U's for (M-Asgn), (M-Field), (M-Call).

3.1 The three tracked shapes

Each shape below is pinned by `tests/expect-error/unsound-refinement.ts` (cases 1–3); all three compile without error today, re-confirmed by running the checker at the pinned revision.

Shape (a) — #597: side-effecting call between test and use.

```
declare function sideEffect(): void;
function f(x: { f: number | null }): void {
  if (x.f !== null) {
    sideEffect();          // (M-Call): may write x.f through an alias
    const n: number = x.f; // accepted; unsound
  }
}
```

R-EqField narrows the object branch's field to $f.\mathsf{get} = \texttt{number}$ at branch entry — sound there. The call is an (M-Call) event: x's object is reachable from the caller of f (it is a parameter, hence aliased), so σ_H may step. The checker's update for the call is the identity, which is *not* adequate for (M-Call): the failing hypothesis is the fixity of σ_H — precisely assumption **L-PURE** read as "the snapshot taken by the snapshot Object Interface is still current at the use". No lemma of the main paper is wrong; its scope boundary ("one-step") is real.

Shape (b) — #599: captured local mutated via an invoked lambda.

```
function f(): void {
  let x: string | null = "hello";
  if (x !== null) {
    const reset = () => { x = null; };
    reset();               // (M-Call) performing (M-Asgn) on captured x
    const s: string = x;   // accepted; unsound
  }
}
```

Here the mutation is not foreign: the checker *typechecks the assignment* `x = null` while checking the lambda body. The event is visible, yet the update applied to the branch environment is the identity — Section 4 shows the assignment lands in the lambda's scope frame, which is discarded. The failing hypothesis is again **L-PURE** at the `reset()` boundary; but unlike shape (a), an exact (not conservative) update was available and thrown away.

Shape (c) — #598: field reassignment inside a refined branch.

```
function f(x: { f: number | null }): void {
  if (x.f !== null) {
    x.f = null;            // (M-Field), checked against set-type: passes
    const n: number = x.f; // accepted; unsound (tsc rejects, TS2322)
  }
}
```

A pure (M-Field) event with no call and no alias needed: the write is syntactic and checked — against the field's set-type (`number | null`, untouched by narrowing), so it passes — but the update applied to $\Gamma(x)$ is the identity, leaving the stale $f.\texttt{get} = \texttt{number}$. The failing hypothesis is the **L-MUT** note's "membership semantics (get-only) is sound for the snapshot reads refinement performs": after the write the snapshot is stale, so $\sigma'(x) \notin [\![\Gamma(x)]\!]^{\sigma'_H}$. This shape is *not* tsc-parity: tsc's assignment narrowing rewrites the declared flow type of `x.f` to `null` and rejects the next line.
impl: `TypeCheckRefine.sk / evalFieldEqualityType (withGet); TType.sk / TFields::resolveFieldType (isLValue reads set)`

4 What the Implementation Does at Each Mutation Site

Established by code inspection at `9108f339` and, where marked *(probed)*, by running the checker (`make check-file-in-docker`) on minimal programs.

4.1 (M-Asgn) direct assignment to a local: sound flow update

`assign`'s `Var`-LHS arm types the RHS and calls `scope.set(n, ty2, ...)`: the local's flow entry is *replaced* by the RHS type in the current frame. This is the adequate update $U_{\text{M-Asgn}}(\Gamma) = \Gamma[x \mapsto \tau_{\text{RHS}}]$ ($\sigma'_L(x) = v \in [\![\tau_{\text{RHS}}]\!]$ by **L-TC**). In particular a narrowed type is correctly clobbered: `if (x !== null) { x = null; const s: string = x; }` is rejected *(probed)*. Through a loop frame, `set` additionally checks $\tau_{\text{RHS}} <:$ loop-entry type — the back-edge guard that keeps iteration-2 reads justified; narrowed loop-entry types participate: narrowing x to `string` before a `while`, then assigning `null` inside it, is rejected *(probed)*.
impl: `TypeCheckExpr.sk / assign (Var arm, ~line 3016); TypeCheckScope.sk / Scope::set (throughLoop check)`

4.2 (M-Asgn) through a closure: unchecked and invisible

`anonFunction` checks a lambda body in a fresh subscope whose `up` is the creation-site scope, then restores the saved scope *without any merge*: the lambda frame and every flow entry written into it are discarded. Combined with the `Scope::set` asymmetry (writes go to the current frame; the loop-guard subtype check fires only across `isLoop` frames, and lambda frames are not loop frames), assignments to captured locals are (i) never propagated and (ii) never checked against the outer binding's type. Two consequences beyond the tracked issues:

Finding 4 (Captured writes are entirely unchecked)**.** (probed) *The program* `let x: string = "a"; const g = () => { x = 5; }; g(); console.log(x.length);` *compiles without any diagnostic; at runtime* `x.length` *is* `undefined`*. This is stronger than #599: no refinement is involved, and the type confusion is arbitrary (*`number` *into* `string`*), not merely a stale narrowing. tsc rejects the assignment (TS2322). Verified distinct from every tracked issue (#597–#599, #585) and confirmed with a runtime witness; issue draft prepared:* `issues/captured-local-writes-unchecked.md`*. Ledger row proposed in Section 7.*

Finding 5 (Closure bodies read narrowings that are stale at invocation)**.** (probed) *Because lookup walks the whole* `up` *chain (**L-LOCAL**'s captured-local half), a lambda body checked inside a refined branch* sees the narrowed type*:* `let x: string | null = "hi"; if (x !== null) { const f = () => x.length; x = null; f(); }` *compiles clean and crashes at runtime. The narrowing was valid when the closure was* created*, not when it* runs*. tsc handles this by re-widening, inside closure bodies, any* `let` *that is assigned anywhere in the function. This is the dual of shape (b): (b)*

is a stale write-out, this is a stale read-in. Confirmed with a runtime witness (a `TypeError` *reading* `.length` *of* `null`*); verified not covered by #599's write-out text; issue draft prepared:* `issues/stale-closure-reads-narrowing.md`*.*

impl: `TypeCheckExpr.sk / anonFunction (∼line 2850: scope save/restore, no popScope); TypeCheckScope.sk / maybeGetRefHelper`

4.3 (M-Field) syntactic field write: checked, but no update

For `x.f = e` (with `x` not a tracked growable local object), `assign` falls to its default arm: it types the LHS in lvalue mode — `dot` with `AKLValue`, which resolves the field via `resolveFieldType(.., isLValue=true)` = the field's set-type, intersected across members — checks $\tau_{\text{RHS}} <:$ that, and returns. *It never touches the scope entry for* `x`: there is no `scope.set` on this path, so a narrowed f.get survives its own overwrite (shape (c)). The narrowing machinery and the write path even share the data: narrowing rewrote f.get via `withGet` and left f.set declared, which is exactly why the write still typechecks. (The separate `assignLocalObject` path for growable local objects *does* update the scope, by intersection — a growth mechanism, not an invalidation one, and out of scope here.)

impl: `TypeCheckExpr.sk / assign (default arm), dot (AKLValue); TType.sk / TFields::resolveFieldType`

4.4 (M-Call): no invalidation whatsoever

No call, `new`, `await`, or method invocation touches `tcenv.scope`. The complete set of scope writers in the checker is: `scope.set` in the refinement primitives, in `assign`'s `Var` arm, in `assignLocalObject`, and in the `this`-retyping after `super(...)`; plus the structured merges (`mergeScope`/`popScope`/`popScopeReplace`). Call typing is absent from that list, so shapes (a) and (b) follow mechanically: the identity update is applied at every (M-Call).

impl: `TypeCheckRefine.sk; TypeCheckExpr.sk / assign; TypeCheckStmt.sk (grep: no scope writes in call paths)`

5 Design Space

Notation: for a function under check, let C be the set of locals captured by at least one closure, $W \subseteq C$ those additionally assigned *anywhere in the function* (matching tsc's discipline and the stale-read witness, whose write is in the enclosing body, not a closure; both computable syntactically in one pre-pass), and say a local is *heap-visible* if its value is an object that may be aliased — conservatively: any object-typed local other than a fresh, never-escaping literal. $\mathsf{havoc}_S(\Gamma)$ re-widens the entries of S: whole-variable narrowings back to the binding's declared type, and field-level narrowings by restoring each narrowed f.get (mechanisms in Section 6).

5.1 Option (i): havoc-on-call

At every (M-Call) boundary apply $\mathsf{havoc}_{W \cup \{\text{field narrowings of heap-visible locals}\}}$. *Soundness:* adequate for (M-Call) by construction — the havocked entries over-approximate every value the event can install (captured writes are bounded by the declared type *once Finding 4 is fixed*; field writes by set-types via **L-MUT**), and unhavocked entries are unreachable by the event (Definition 2: uncaptured locals cannot be written by a call). *Precision cost:* the breaking shape is "narrow once, then several statements each containing a call and a later use of the narrowing". Spot-checks against the repository corpus at the pinned revision:

- `examples/src/type_refinement_examples.ts` (`processResponse`): `response.kind === "success"` narrowing used once, *inside the argument* of the following `console.log` — argument evaluation precedes the call, so a call-boundary havoc does not break it. Survives.
- `examples/src/deck.ts` (display loop): `card` narrowed by `!== undefined`, used only in the argument of one `console.log`. Survives.
- `examples/src/runtime_demo.ts` / `builtins_documentation.ts`: `aliceAge !== undefined` then `aliceAge.toString()` inside one call argument. Survives — but note `aliceAge` is not captured and not object-typed, so option (i) as scoped above would not havoc it anyway; only a cruder "havoc everything" would, and even that survives these single-use sites.
- Breaking witness (crude variant): any `if (x) { log(x.a); log(x.b); }` with `x` a narrowed *field path or captured local* — the first `log` havocs, the second use fails. `examples/src/try_catch.ts` (uses `e.message` after an earlier statement in the branch) is the nearest real instance; it survives the scoped option (i) only because the intervening statements contain no call.

The corpus tolerates the *scoped* havoc well: narrowed values are overwhelmingly consumed before or inside the first following call. The cost concentrates on ADT-style field narrowings used across several statements with interleaved calls — common in real code, rarer in this repo.

5.2 Option (ii): effect / purity tracking

Havoc only at calls whose callee may write the relevant location. SKJS already builds a call graph during checking (`CGNode`, `recordCalls`), so the skeleton exists; what is missing is (1) a per-function summary "writes captured local x" / "writes some field f through some alias", computed to a fixpoint over the call graph including indirect calls through function-typed values — which in practice degrades to "any unknown callee may write anything", (2) summaries for `declare`d ambient functions, which have no bodies — so either an effect annotation surface is added to the language (a major design commitment tsc has repeatedly declined) or ambient calls havoc everything, and (3) soundness of the summary under first-class functions, requiring the call graph itself to be sound, which it is not required to be today (it exists for diagnostics/codegen ordering). *Verdict:* strictly dominates (i) in precision when it fires, but its sound version collapses to (i) at every ambient or indirect call — the overwhelmingly common case (`console.log` is ambient). Not recommendable as the primary mechanism; viable later as a precision overlay on top of (i)/(iii).
impl: `Reachability.sk / CGNode; TypeCheckEnv.sk / recordCalls`

5.3 Option (iii): capture-based invalidation

Precompute W (captured-and-assigned locals). Then: (1) at every (M-Call) boundary, havoc_W; (2) inside closure bodies, do not let narrowings of W-locals flow in — resolve them at their declared type (fixing Finding 5); (3) check captured writes against the declared type (fixing Finding 4, and making "declared type" a sound havoc target). This is sound for *whole-local* narrowings: an (M-Call) can write exactly the W-locals (Definition 2), and each write is now bounded by the declared type. It deliberately does *not* address field narrowings (shape (a)) — the heap is aliased regardless of capture. tsc implements a cousin of this policy in restricted positions (narrowings of `var`s and of `let`s assigned in nested functions are dropped inside closure bodies, and loop-carried narrowings are reset), but performs no call-boundary invalidation at all. *Precision cost:* negligible in the corpus

— no repository example narrows a captured-and-assigned local and then calls; the pinned #599 reproducer is the canonical instance and is exactly what should break.

5.4 Option (iv): status quo (documented tsc-parity unsoundness)

Keep everything, document L-PURE as violated (done), pin the reproducers (done). This is genuine tsc parity for shapes (a) and (b) — tsc accepts both — but *not* for shape (c) or Finding 4, both of which tsc rejects. Status quo is therefore not a coherent resting point: two of the five behaviors are strictly-more-unsound-than-tsc bugs, not policy choices.

5.5 Recommendation

Policy 6 (Adopted invalidation policy)**.** *Layered, in decreasing order of urgency:*

1. ***P1 (bug fix):*** *check captured writes against the captured binding's declared type (Finding 4); concretely,* `Scope::set` *applies its subtype check when the lookup crossed a* function *frame, exactly as it already does for loop frames.*

2. ***P2 (#598):*** *re-widen the narrowed field getter at every syntactic field write (rule in Section 6).*

3. ***P3 (#599 + Finding 5):*** *option (iii) capture-based invalidation for W-locals: havoc at calls, block narrowing inflow to closure bodies.*

4. ***P4 (#597):*** *option (i) scoped to field-level narrowings of heap-visible locals, applied at (M-Call) boundaries. This is the only layer with a real precision cost; if product priorities delay it, the deviation is exactly tsc's, and L-PURE's ledger row shrinks to this one named residue rather than three shapes.*

Lemma 7 (Preservation across mutation events — replaces one-step scoping at mutation sites)**.** *Assume the main paper's axioms (**L-UNION**, **L-INTER**, **L-EXP**, **L-MONO**, **L-TC**, **L-SUB**) and the well-formedness condition $[\![f.\mathsf{get}]\!] \supseteq$ the values a read of f can yield after an accepted write (Obligation 9). Under Policy 6 (P1–P4), for every checker path point t with environment Γ_t and every event $\sigma \xrightarrow{\mu} \sigma'$ occurring between t and the next point t': $\sigma \models_\rho \Gamma_t$ implies $\sigma' \models_\rho \Gamma_{t'}$, where $\Gamma_{t'}$ is the environment produced by the policy's update for μ (Definition 3): $U_{\text{M-Asgn}} = [x \mapsto \tau_{\text{RHS}}]$ with the P1 declared-type check for captured writes; $U_{\text{M-Field}}$ the P2 re-widening; $U_{\text{M-Call}} = \mathsf{havoc}_W \circ \mathsf{havoc}_{\text{fields}}$ (P3, P4). Consequently branch soundness extends from branch entry to every use in the branch, by induction over the events on the path.*

Proof sketch. Case (M-Asgn): **L-TC** for the RHS; for captured writes the P1 check bounds the written value by the declared type, which is what havoc later restores. Case (M-Field): Lemma 8. Case (M-Call): the event can write only W-locals and the heap (Definition 2); havoc_W restores declared types, sound by the P1 bound; $\mathsf{havoc}_{\text{fields}}$ restores get-types whose adequacy is Obligation 9; entries outside both havoc sets denote locations the event cannot write, so their satisfaction transports unchanged. □

6 The Re-Widening Rule for Field Writes (#598)

Rule W-Field. On `x.f = e` with RHS type τ_{RHS} (after the existing $\tau_{\mathrm{RHS}} <: \bigsqcap f.\mathsf{set}$ check), update `x`'s scope entry by the memberwise primitive $\mathsf{refine}^{g,\mathsf{None}}$ with mapper: non-object members unchanged; each object branch β with a declared field f becomes

$$\beta[f.\mathsf{get} \mapsto \mathsf{union}(f.\mathsf{get},\ \tau_{\mathrm{RHS}}),\quad f.\mathsf{opt} \mapsto f.\mathsf{opt}]$$

and branches without f (or reached only via an index) are unchanged. Optionality is *kept*: clearing it would be sound for data fields (the write makes f present) but unsound for a branch whose write went through a setter-less index — keep the conservative reading.

This reuses the exact machinery of `narrowField`: the same `setLocalFilterMap` walk, the same `updateField`/`withGet` write-back — with union where narrowing used inter/rmc, and with the branch-dropping arm removed (a write never refutes a branch). No new representation is needed.

impl: `TypeCheckRefine.sk / narrowField, setLocalFilterMap (mechanism to reuse); TypeCheckExpr.sk / assign (default arm | the missing scope.set site)`

Lemma 8 (W-Field is adequate for (M-Field))**.** *If $\sigma \models_\rho \Gamma$ and $\sigma \xrightarrow{o.f:=v} \sigma'$ with $v \in [\![\tau_{\mathrm{RHS}}]\!]$ and $o = \sigma_L(x)$, then $\sigma' \models_\rho U(\Gamma)$ for the W-Field update U — provided f is data-backed in o* and *no other local's recorded type views the same object through a wider* set*-type for f (cross-alias get/set coherence). The latter hypothesis is not discharged here: a declared alias `const y: {f: number} = x` after narrowing `x` admits a legal write `o.f := null` that U leaves unrecorded on y — a hole shared with tsc (property covariance) and strictly stronger than Obligation 9; absent a coherence invariant, the "*PROVED *for data fields" trajectory is a tsc-parity residue, not an unconditional discharge.*

Proof. Only x's entry changes and only o's f-property changed in σ_H. Let β be the object branch with $o \in [\![\beta]\!]$. If β declares f: every constraint of β except the f-value constraint is insensitive to the event; the new value v satisfies the new get-type by $v \in [\![\tau_{\mathrm{RHS}}]\!] \subseteq [\![\mathsf{union}(f.\mathsf{get}, \tau_{\mathrm{RHS}})]\!]$ (**L-UNION**). If β does not declare f but an index covers it, the index constraint on the new value holds because the accepted write checked τ_{RHS} against the resolved set-type, which for an index-covered key includes the index's set-type, and the well-formedness of Obligation 9 carries it to the get side. Entries of other locals may alias o: their branches' $f.\mathsf{get}$ must also admit v — this is *not* guaranteed by W-Field (it updates only x) and is exactly why P4 additionally havocs field narrowings of other heap-visible locals at writes-through-aliases; for the syntactic single-local write of #598, aliasing within the same function without an intervening call also requires havocking *other* locals' narrowed f — the implementation should apply W-Field to every local whose type carries a narrowed field named f, a cheap over-approximation. □

The accessor situation. At the pinned revision SKJS does not represent `get`/`set` accessor members as get/set fields at all: the parser records an `isGetSet` marker (`FGet`/`FSet`) but the type checker never consults it, so a getter `get p(): T` is typed as a *method* `p(): T` (a setter likewise, then rejected for lacking a return type). That method-typing is itself unsound in the *call* direction — `const n: number = c.p()` type-checks but throws at runtime, and tsc rejects it (TS6234); registry row ACCESSOR-METHOD (issue draft `getter-typed-as-method.md`), a known limitation (tests `callgraph-unsupported/getter-calls.ts`). For re-widening the consequence is that there are no coercing-setter fields: a getter read is never a narrowable value (it is a function), and

the only fields with genuinely distinct `get/set` are synthetic index signatures (independent get/set variables) and readonly fields (`set`$=\bot$), neither of which runs user code on a write. So the W-Field rule's union against τ_{RHS} carries no accessor residue at present; its only residue is the cross-alias one of Lemma 8.

The accessor caveat, forward-looking. *If* accessor members were represented as get/set fields, writing `o.f = e` would run a coercing setter, which could store — and a subsequent get return — any value of the *declared* get-type, which union(narrowed get, τ_{RHS}) need not contain. The sound re-widening would then be union(f.get, f.set), which contains the declared get-type whenever declared-get $\subseteq$ set holds (Obligation 9); tsc's τ_{RHS} narrowing is unsound for coercing accessors in the same way. Because `TField` records only `get/set/access` and not accessor-ness, the union(f.get, f.set) form would be the sound option expressible even then — and supplying accessor-ness, so the checker builds get/set accessor fields, is the same change that closes ACCESSOR-METHOD. We therefore recommend the τ_{RHS} variant now (sound today up to the cross-alias residue, and matching tsc), revisiting the accessor case if and when accessor fields are added.

Obligation 9 (Get/set coherence)**.** *For every well-formed field, any value readable from the field after an accepted write lies in* $[\![f.\text{get}]\!]$ *read at the field's* declared *(un-narrowed) type. For data fields this is immediate (the read returns the written value, bounded by* $f.\text{set} = f.\text{get}$ *declared); for accessors it needs the declared-get* $\subseteq$ *set check, which SKJS should enforce at declaration (tsc's TS2380 lineage). Status:* OPEN*; needed by Lemma 7's field case and by exit (1) above.*

7 Status and Registry Impact

The authoritative, current statuses are maintained in the registry (`docs/ledger.yaml`), rendered as the main paper's Assumptions and Proof Obligations tables; this paper is cited there as evidence. The registry rows it supports:

Row	Statement	Status
L-LOCAL	Module-level bindings are never narrowed; function-level locals are, including captured ones.	PROVED
L-PURE	Evaluating a condition does not mutate the narrowed locals.	VIOLATED
L-CAPT	Assignments to captured locals are checked against the binding's declared type.	VIOLATED
L-GETSET	Get/set coherence of accessor fields.	OPEN
ACCESSOR-METHOD	Accessor (`get/set`) members must be typed as fields, not methods; today a getter is a call signature, so `c.p()` type-checks and throws.	VIOLATED

Imports (review contract). The named statements this paper assumes from elsewhere. A reviewer treats each as an axiom, checking citations against the registry's canonical statement; no other paper's internals are in scope:

Row	Canonical statement	Owner	Status
L-UNION	`union` over-approximates set union (all levels).	B	PROVED
L-INTER	`inter` over-approximates set intersection (the containment clause).	B	PROVED
L-EXP	Expansion preserves or enlarges semantics (modes `None`/Covariant); residual instances keep theirs.	C	PROVED
L-MONO	Refinement-time state evolutions preserve membership in already-installed types.	E	PROVED
L-SUB	`Clauses::isSub` acceptance implies semantic inclusion under the current state.	D	OPEN
L-TC	Checked expressions inhabit their checked types.	main	ASSUMED
L-MUT	Field writes are governed by `set`-types; get-only membership is sound for snapshot reads.	main	ASSUMED

What this paper establishes per row, and how the specified fixes change the registry when they land:

Row	Established here / fix impact	Trajectory when the fix lands
L-PURE	Fix specified: Policy P1–P4. Lemma 7 discharges the *caller-path* shapes (a) #597 by P4 and (b) #599 by P3. Three residues it does *not* yet cover: the closure *stale-read* (Finding 5) is closed by P3's static inflow rule, not by the preservation induction, and needs a separate closure-body lemma; a narrowed *field* read inside a closure is closed by no layer (a witnessed shape, issue draft `stale-closure-reads-narrowing.md`); and implicit invocations (getters/coercions/iterators) are outside the (M-Call) event model, so P3/P4 do not fire there.	the caller-path shapes close once P3/P4 land; the closure-body read and event-model residues remain
L-MUT	#598 has a specified re-widening rule (W-Field, adequate by Lemma 8 for data fields); accessor residue gated on get/set coherence (Obligation 9).	data fields discharged once W-Field lands
L-LOCAL	The captured-local half made precise: closure bodies read enclosing narrowings valid at creation, not invocation (Finding 5); P3 blocks inflow for captured-and-assigned locals.	captured half governed by L-PURE/L-CAPT
L-CAPT	Made precise, with a witness (Finding 4): the captured write is unchecked and discarded; fix is P1; issue draft `captured-local-writes-unchecked.md`.	closes once P1 lands
L-GETSET	Introduced here (Obligation 9): prerequisite for the sound accessor variant of W-Field.	gated on the get/set coherence check

Recommended pinned tests. Extend `tests/expect-error/unsound-refinement.ts` (cases 1–3 already pin shapes (a)–(c)) with: case 4, Finding 4 (captured write of a wrong-typed value, no refinement involved); case 5, Finding 5 (closure reads a narrowing, local reassigned, closure invoked). Each policy layer that lands flips its cases from expect-clean to expect-error, forcing the

corresponding passage of this paper and of the main paper's ledger to be updated in the same change — the maintenance discipline of the main paper's Appendix B. Additionally pin the two *sound* behaviors this paper leans on, so regressions surface: direct-assignment clobbering (`x = null` after narrowing rejects the next use) and the through-loop back-edge guard (assignment inside a loop checked against the narrowed loop-entry type) — both verified by probe at the pinned revision but not currently pinned by any test.

Failed attempts and negative findings, for the record. (1) A pure "havoc everything at every call" baseline was considered and rejected without implementation: spot-checks (Section 5) show it would already break single-expression uses like `aliceAge.toString() + aliceAge.toString()` only under sub-expression granularity, but its statement-level version survives the corpus surprisingly well — the reason to reject it is principle (it havocs locals a call provably cannot reach), not corpus breakage. (2) An attempt to justify clearing optionality in W-Field failed on the index-covered-write case and was withdrawn (rule keeps optionality). (3) An attempt to state Lemma 8 without the aliased-locals havoc failed — the two-locals-one-object counterexample is noted in its proof; the lemma is honest only with the "apply W-Field to every local carrying a narrowed f" clause. (4) Effect tracking (option ii) was evaluated and found to collapse to havoc at every ambient/indirect call; recorded so it is not re-proposed without an annotation surface.